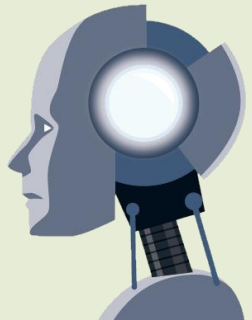
04

운영체제와 활용

단원 목차

4.1 운영체제 개요
4.2 운영체제 관리
4.3 운영체제의 분류
4.4 다양한 운영체제의 종류
4.5 윈도우 운영체제 활용



운영체제 개요

운영체제(OS: Operating System)는 컴퓨터의 주기억장치 내에 상주하면서 사용자와 컴퓨터 시스템 사이의 인터페이스를 담당

- **운영체제 OS(Operation System) 정의**
  - 컴퓨터 시스템의 전반적인 동작을 제어하고 조정
    - **사용자와 컴퓨터 시스템 사이에 위치**
  - 사용자에게 편리성을 제공
    - **한정된 시스템 자원을 효율적으로 사용할 수 있도록 관리**
      - **중앙처리장치(CPU)**
      - **주기억장치, 입출력장치**
      - **파일 시스템 등의 컴퓨터 자원을 관리**

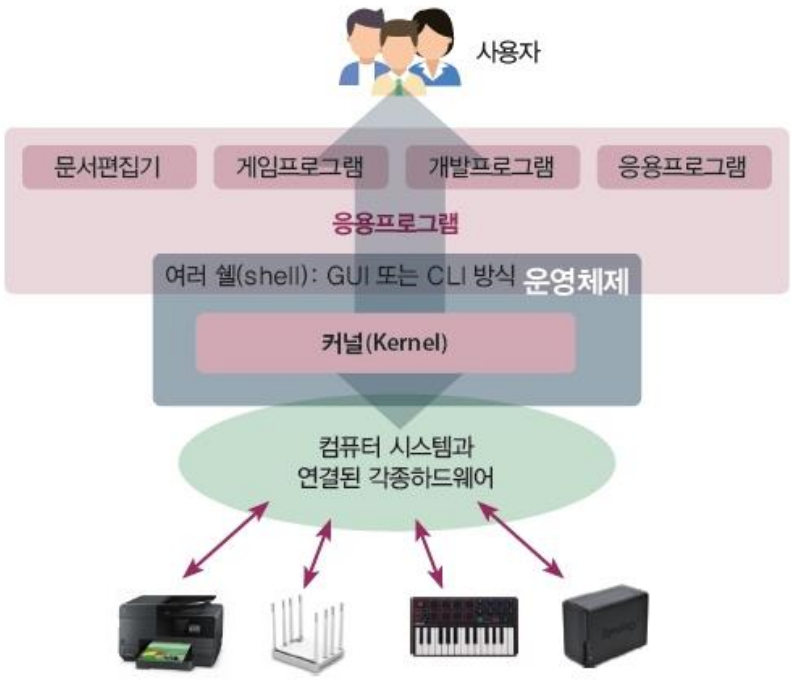


그림 5.1 ▸ 운영체제 정의

- **운영체제 종류**
  - 개인용컴퓨터와 중대형 컴퓨터

표 5.1 운영체제 종류

개인용 컴퓨터	중대형 컴퓨터	스마트폰 및 기기
• MS-DOS • Windows 계열 • OS/2 • Linux 계열 • 맥OS	• Unix 계열 • Windows Server 계열 • VMS	• iOS • Android • Windows

운영체제 목적과 기능

운영체제의 목적은 컴퓨터 시스템의 자원을 편리하게 사용할 수 있는 환경을 제공하고 컴퓨터 시스템의 자원들을 효율적으로 관리하여 시스템의 성능을 최적화

- **목적**
  - 처리능력(Throughput) 향상
  - 반환시간(Turnaround Time) 단축
  - 사용 가능도(Availability) 극대화
  - 신뢰도(Reliability) 보장

운영체제 목적과 기능

운영체제의 목적은 컴퓨터 시스템의 자원을 편리하게 사용할 수 있는 환경을 제공하고 컴퓨터 시스템의 자원들을 효율적으로 관리하여 시스템의 성능을 최적화

- **기능**
  - 컴퓨터의 기능을 사용자가 잘 활용하도록 돕고
  - 하드웨어가 성능을 잘 발휘하도록 관리

기능	설명
프로세스 관리	프로세스 생성·스케줄링·종료
기억장치 관리	주기억장치 할당·가상 메모리 관리
파일 시스템 관리	파일 생성·삭제·접근 제어
입·출력 관리	I/O 장치 제어·버퍼링
보호·보안	자원 접근 통제·오류 복구

사용자 인터페이스 방식

사용자 인터페이스 방법에는 크게 명령행 인터페이스 방법과 그래픽 사용자 인터페이스 방법

- **명령행 인터페이스(CLI) 방식**
  - 운영체제가 제공하는 기능을 키보드 입력을 통해 사용하는 인터페이스 방법
    - **MS 도스(MS-DOS)**
    - **유닉스(UNIX)의 여러 쉘(Shell)**
    - **윈도우즈의 명령 프롬프트(Command Prompt)**
- **그래픽 사용자 인터페이스(GUI) 방식**
  - 제록스(Zerox) 사의 폴로 알토(Palo alto) 연구소에서 1970년대 처음 개발
    - **애플의 맥 OS**
    - **유닉스의 X 윈도우**
    - **MS 윈도우**

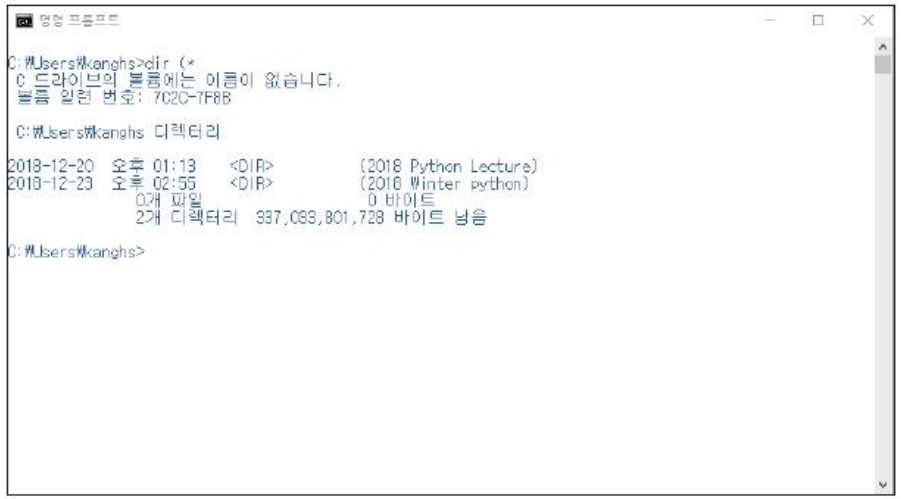


그림 5.2 ▸ 윈도우 계열의 명령 프롬프트

그림 5.3 ▸ 윈도우 11의 GUI(www.apple.com)

운영체제 관리

운영체제의 가장 중요한 기능은 시스템의 여러 자원을 관리하는 것

- **관리 대상**
  - 기억장치, 프로세스, 입출력장치, 파일
- **기억장치 관리 기능**
  - 어느 프로세스를 주기억장치에 적재할지 결정해 적절한 주기억장치에 프로세스를 적재
  - 프로세스가 더 이상 주기억장치를 필요로 하지 않으면 다시 주기억장치를 회수
  - 종류
    - **단일 프로그램 방식과 다중 프로그램 방식**

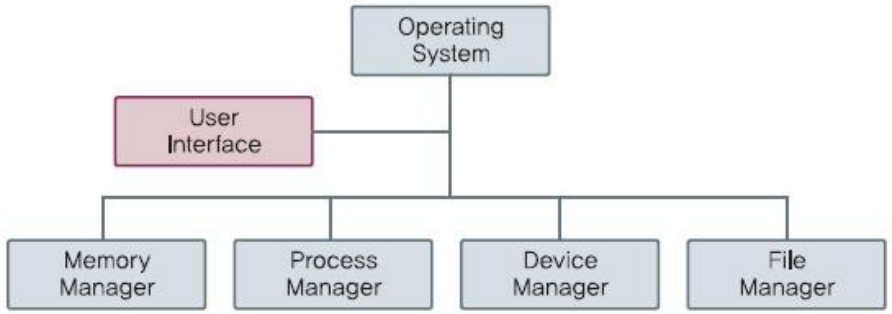


그림 5.4 ▸ 운영체제 관리자

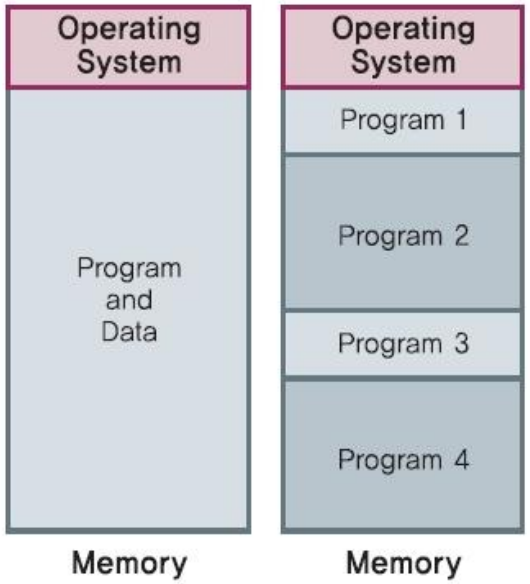


그림 5.5 ▸ 단일 프로그램 방식과 다중 프로그래밍 방식

프로세스 관리

메모리에 적재되어 CPU(프로세서)의 처리를 기다리거나 CPU에 선택되어 처리가 수행되는 프로그램

- **기능**
  - 프로세스의 생성과 삭제
  - 프로세스의 중지와 계속
  - 프로세스의 동기화 등의 기능을 수행
  - 프로세스의 정의
    - **시작했지만 아직 종료되지 않은 프로그램으로 주기억장치에 적재되어 있는 프로그램**
- **프로세스의 상태**
  - 준비(Ready), 대기(Waiting), 실행(Running)의 3가지
  - 프로세스 스케줄러에 의해 관리

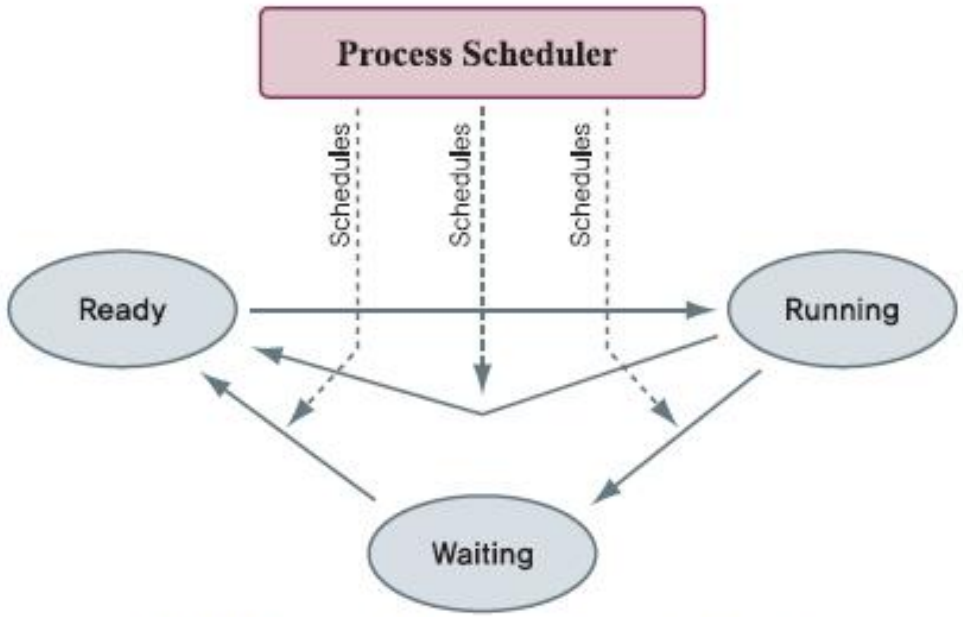


그림 5.6 ▸ 프로세스 관리자와 프로세스 상태

프로세서, 프로세스, 프로그램

- **프로세서(Processor)란**

  - 중앙 처리 장치를 의미하며, 컴퓨터에 들어가는 부품 중 하나인 CPU가 프로세서

- **프로세스(Process)란**

  - 메모리에 적재되어 실행 중이거나 대기 중인 프로그램을 프로세스

- **프로그램(Program)란**

  - 컴퓨터에서 사용자가 원하는 목적을 달성하기 위해 목적에 맞게 특정 작업을 수행하는 일련의 명령어들의 집합

스케줄링

- **스케줄링**
  - 프로세스가 생성되어 실행될 때 필요한 시스템의 여러 자원을 해당 프로세스에게 할당하는 작업
  - 대기 시간은 최소화하고 최대한 공평하게 처리하는 것을 목적으로 함
- **스케줄링의 종류**
  - 선점(preemptive) 스케줄링
  - 비선점(non-preemptive) 스케줄링

선점 스케줄링

- 하나의 프로세스가 CPU를 차지하고 있을 때, 우선순위가 높은 다른 프로세스가 현재 프로세스를 중단시키고 CPU를 점유하는 스케줄링 방식
- 비교적 응답이 빠르다는 장점이 있지만, 처리 시간을 예측하기 힘들고 높은 우선순위 프로세스들이 계속 들어오는 경우 오버헤드 초래
- 종류 : Round Robin, SRT, 선점 우선순위, 다단계 큐, 다단계 피드백 큐 등

비선점 스케줄링

- 이미 할당된 CPU를 다른 프로세스가 강제로 빼앗아 사용할 수 없는 스케줄링 기법
- 프로세스가 CPU를 할당 받으면 해당 프로세스가 완료될 때까지 CPU를 사용하지 않음
- 종류 : FCFS, SJF, 우선순위, HRN, 기한부 등

장치와 파일 관리

입출력 장치와 파일의 관리

- **장치관리 기능**
  - 입출력 장치의 할당과 회수 등의 기능
  - 프로세스의 입출력
    - **인터럽트(interrupt)을 이용하여 처리**
  - 장치 제어기(Device Driver)
    - **장치관리자와 하드웨어인 입출력장치 사이의 인터페이스를 담당하는 프로그램**
- **파일관리 기능**
  - 보조기억장치에 저장되는 파일을 관리
    - **파일의 접근을 제어**
    - **파일의 생성, 삭제, 수정을 감독**
    - **파일을 디스크의 어디에 저장할 지를 감독**
    - **파일을 여러 사용자가 공유하는 기법을 제공**
    - **폴더의 구조를 제공하고 관리**

그림 5.7 ▸ 장치 제어기 연결 구조

그림 5.8 ▸ 파일 관리자의 역할

인터럽트

- 프로그램을 실행하는 도중에 예기치 않은 상황이 발생할 경우 현재 실행중인 작업을 중단하고 발생된 상황을 처리한 후 다시 실행중인 작업으로 복귀하는 것
- 인터럽트의 종류 : 외부 인터럽트, 내부 인터럽트, 소프트웨어 인터럽트

인터럽트

외부인터럽트

- **전원 이상 인터럽트(Power fail interrupt) : 말그대로 정전, 파워 이상 등**
- **기계 착오 인터럽트(Machine check interrupt) : CPU의 기능적인 오류**
- **외부 신호 인터럽트(External interrupt)**
  - 타이머에 의한 인터럽트 : 자원이 할당된 시간이 다 끝난 경우
  - 키보드로 인터럽트 키를 누른 경우 : 대표적으로 Control + Alt + Delete
  - 외부장치로부터 인터럽트 요청이 있는 경우
- **입출력 인터럽트(I/O Interrupt)**
  - 입출력장치가 데이터 전송을 요구하거나 전송이 끝나 다음 동작이 수행되어야 할 경우
  - 입출력 데이터에 이상이 있는 경우

인터럽트

내부인터럽트

- **잘못된 명령이나 잘못된 데이터를 사용할때 발생하며 Trap이라 부른다.**
- **프로그래 검사 인터럽트(Program check interrupt)**
  - Division by zero
  - Overflow/Underflow
  - 기타 Exception

인터럽트

소프트웨어 인터럽트(SVC : SuperVisor Call)

- 사용자가 프로그램을 실행시키거나 감시프로그램(Supervisor)을 호출하는 동작을 수행하는 경우
- 소프트웨어 이용중 다른 프로세스를 실행시키면 시분할 처리를 위해 자원 할당 등의 동작이 수행된다.

인터럽트 동작 순서

1. **인터럽트 요청**
2. **프로그램 실행 중단 : 현재 실행중이던 Micro operation 까지 수행한다.**
3. **현재의 프로그램 상태 보존 : PCB(Process Control Block), PC(Program Counter) 등**
4. **인터럽트 처리루틴 실행 : 인터럽트를 요청한 장치를 식별한다.**
5. **인터럽트 서비스 루틴 실행 :**
    1. 인터럽트 원인을 파악하고 실질적인 작업을 수행한다.
    2. 처리기 레지스터 상태를 보존한다.
    3. 서비스 루틴 수행 중 우선순위가 더 높은 인터럽트가 발생하면 또 재귀적으로 1~5를 수행한다.
6. **상태복구 : 인터럽트 발생 시 저장해둔 PC(Program counter)를 다시 복구한다.**
7. **중단된 프로그램 실행 재개 : PC의 값을 이용하여 이전에 수행중이던 프로그램을 재개한다.**

인터럽트 우선순위

- 전원 이상(Power fail)
- 기계 착오(Machine Check)
- 외부 신호(External)
- 입출력(I/O)
- 명령어 잘못
- 프로그램 검사(Program Check)
- SVC(SuperVisor Call)

일반적으로 하드웨어 인터럽트가 소프트웨어 인터럽트보다 우선 순위가 높고 내부 인터럽트 보다 외부 인터럽트가 우선 순위가 높다.

초기 운영체제

운영체제라는 형태의 개념을 처음 선보인 것은 1952년 IBM-701 컴퓨터의 사용을 위해 GM(General Motors) 연구소에서 개발한 프로그램

- **초기 운영체제**
  - 프로그래머는 각종 장치의 전면 판의 스위치를 이용하여 직접 프로그램을 주기억장치에 적재하고 해당하는 버튼을 눌러 프로그램을 실행
- **일괄처리방식**
  - 일정 기간 또는 일정량의 자료 처리를 모아 두었다가 한 시점에 순서적으로 일괄 처리 방식
  - 장단점
    - **한 번 시스템을 차지한 자료는 시스템 자원을 독점하여 처리하므로 컴퓨터 시스템을 효율적으로 사용**
    - **CPU가 사용 가능하더라도 다른 처리를 할 수 없는 유휴 시간(idle time)을 갖게 되는 단점**
    - **작업 제출과 작업 완료 사이의 시간(turn around time)이 많이 걸려 작업 결과를 빠르게 확인할 수 없는 단점**

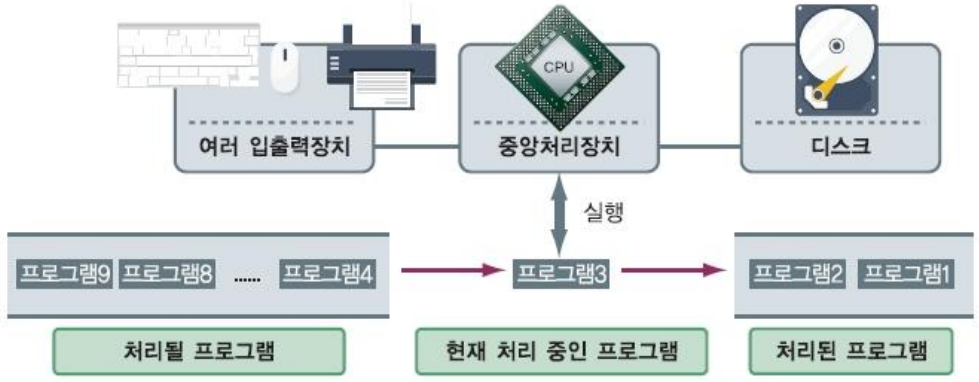


그림 5.10 ▸ 일괄처리 방식

시분할 시스템

1961년에 매사추세츠 공과대학(MIT)에서
호환 시분할시스템(CTSS: Compatible Time Sharing System)을 개발함으로써 개념을 정립

- **방식**
  - 하나의 시스템을 여러 사용자가 단말기 이용
    - **여러 작업을 처리할 때 이용하는 처리 방법**
  - CPU의 이용 시간을 잘게 분할
    - **여러 사용자의 작업을 순환하며 수행**
  - 타임 조각(time slice) 방법, 시분할을 구현
    - **실행 프로그램은 연산처리장치의 할당 시간을 돌아가며 부여 받음**
  - 장점
    - **단말기만 이용할 수 있다면 짧은 작업인 경우엔 수행을 빨리 완료할 수 있는 특징**
    - **중앙처리장치의 유휴 시간을 줄일 수 있음**

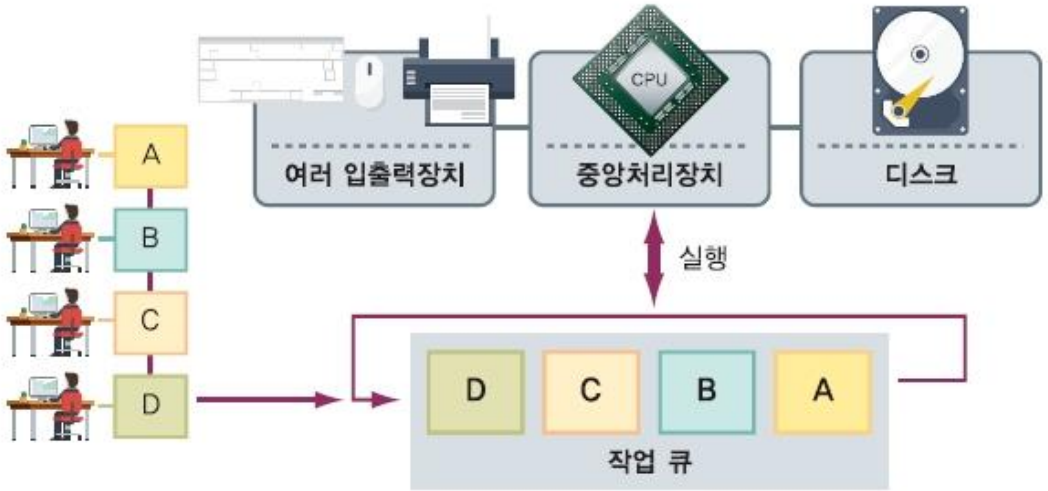


그림 5.11 ▸ 시분할 시스템의 개념

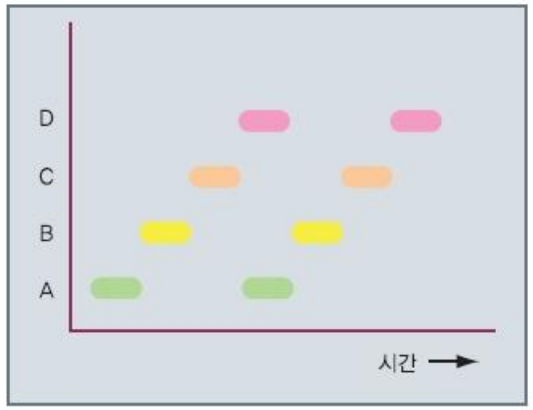


그림 5.12 ▸ 타임 조각

다중 프로그래밍 시스템

2개 이상의 여러 프로그램을 주기억장치에 동시에 저장하고 하나의 CPU로 실행하는 방식

- **여러 프로그램이 동시에 실행되는 것과 같이 보임**
  - 실제로 여러 프로그램이 동시에 수행되는 것이 아니고
    - **한 프로그램씩 순차적으로 돌아가면서 조금씩 수행**
  - CPU의 속도가 매우 빠르므로
    - **우리가 보기에는 동시에 수행되는 것처럼 보일 뿐**
- **기억장치 관리 기법과 CPU 스케쥴링 기법이 필요**
  - 하나의 프로그램이 주변장치의 처리를 기다리는 동안
    - **다른 프로그램이 CPU에서 실행될 수 있게 함으로써 전체적인 시스템의 처리 효율을 높이는 방식**

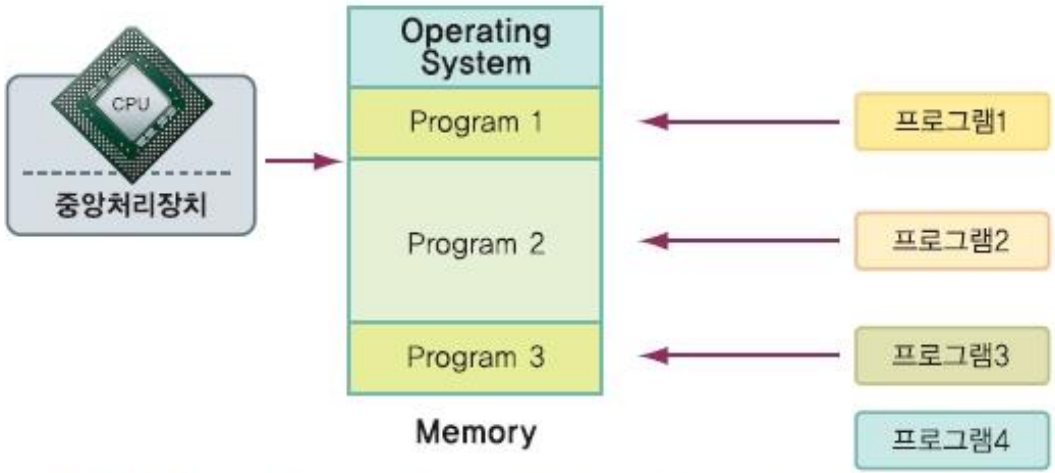


그림 15.13 ▸ 다중 프로그래밍 시스템의 개념(현재는 프로그램 2를 시행 중)

다중 처리 시스템

2개 이상의 중앙처리장치(CPU, processor)를 사용하여 작업을 여러 개로 분담하여 프로그램을 동시에 수행하는 방식

- **특징**
  - 하나의 CPU에 문제가 생기더라도 다른 CPU가 처리를 계속할 수 있으므로
    - **신뢰도를 높일 수 있음**
  - 일반적으로 여러 개의 프로세서가 하나의 운영체제에 의해 관리되고 버스나 기억장치를 공유하여 서로 통신하며 작업을 처리하므로
    - **작업 처리 속도도 빠름**
- **종류**
  - 비대칭적(asymmetric) 다중 처리 방식
    - **별도의 주 처리기만이 운영체제를 수행하며 제어를 담당**
  - 대칭적 다중 처리 방식
    - **모든 처리기들이 하나의 운영체제를 동시에 수행하며 각자의 작업을 수행**

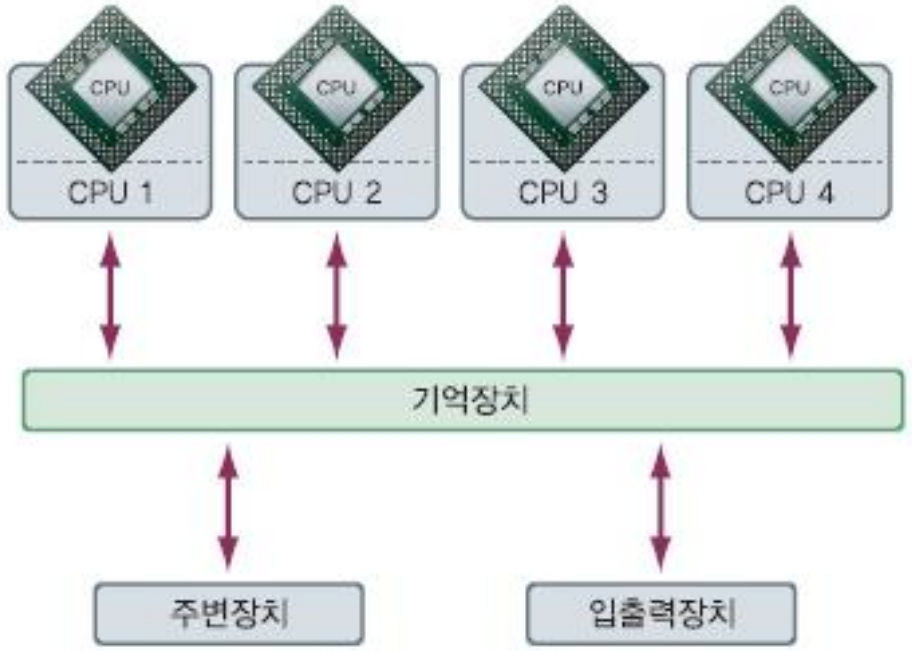


그림 5.14 ▸ 다중처리 시스템의 개념

다중 작업

한 사람의 사용자가 한 대의 컴퓨터로 2가지 이상의 작업(task)을 동시에 처리하거나 프로그램들을 동시에 구동 시키는 기능

- **다중작업**
- **단일작업**
  - 다중작업(Multiprocessing)과는 다르게 한 사용자가 한 번에 한 가지 작업이나 한 프로그램 밖에 실행하지 못하는 시스템
  - MS-DOS 운영체제

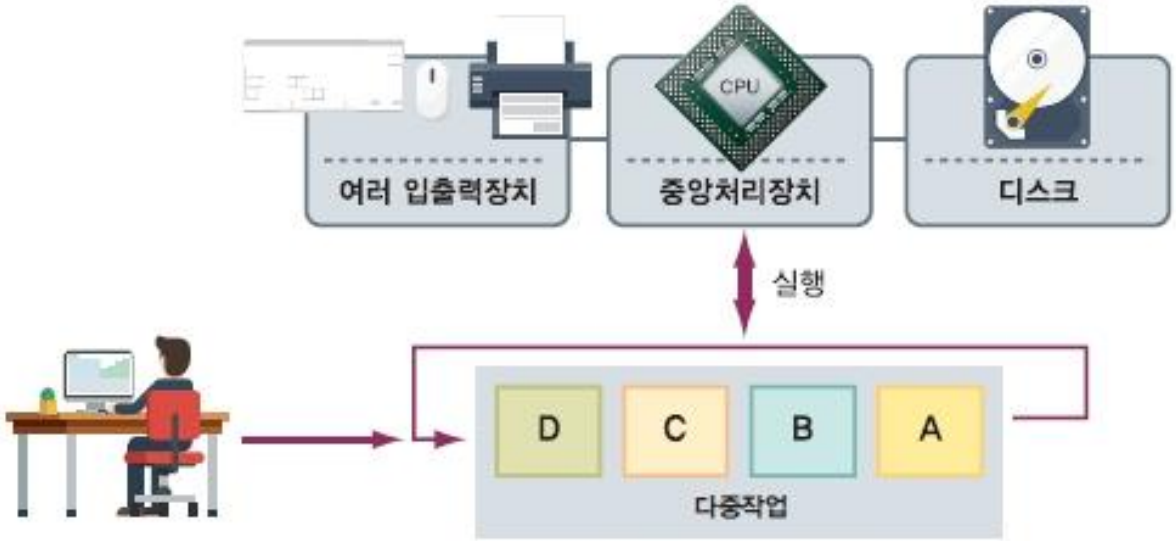


그림 5.15 ▸ 다중작업 방식의 개념

실시간 처리 시스템

처리를 요구하는 작업이 발생할 때마다 지정된 짧은 시간 내에 작업을 처리하여 확실한 응답이나 출력을 보장하는 시스템

- **특징**
  - 실시간 처리 시스템은 자료가 발생한 단말기를 통해 그 처리가 가능하므로 사용자가 편리
  - 처리 시간이 단축되고 처리 비용도 절감
  - 시스템에 오류가 발생하면 심각한 문제가 발생할 수 있으므로 다소 위험성이 있는 시스템
- **예**
  - 미사일 방어 시스템
  - 좌석 예약 시스템
  - 은행의 예금 업무

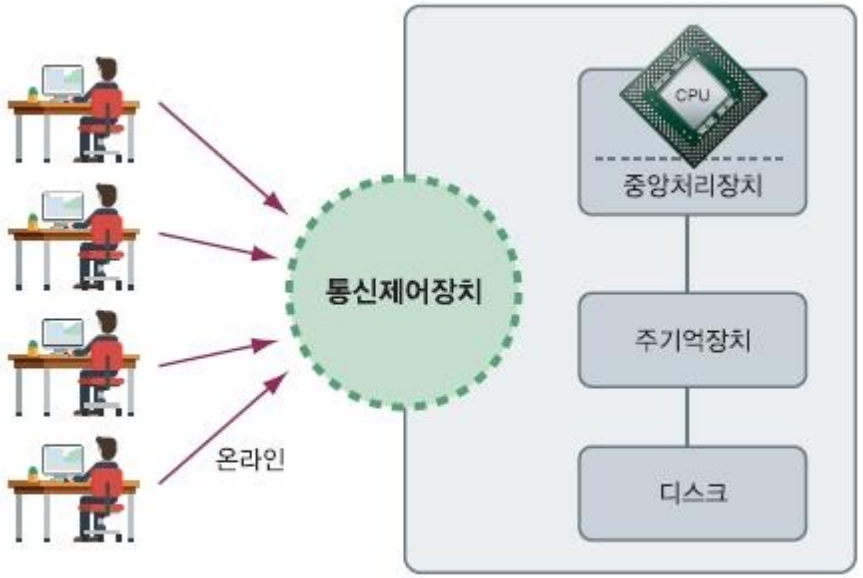


그림 5.16 ▸ 실시간 처리 시스템의 개념

분산 처리 시스템

네트워크를 통해 연결된 여러 자료 저장 장소와 컴퓨터 시스템에 작업과 자원을 나누어 서로 통신을 하면서 일을 처리하게 하는 방식

- **특징**
  - 여러 개의 자료 저장 장소와 시스템을 이용하여 동시에 여러 작업을 수행함으로써 성능이 향상
  - 자료도 복사본을 여러 곳에 유지할 수 있어 신뢰도를 높일 수 있음
  - 네트워크에 새로운 처리 시스템을 추가함으로써 쉽게 시스템 확장도 가능
  - 장점을 잘 살리려면
    - **자료 저장장소와 시스템을 단순히 물리적으로 연결해서는 안 되고**
    - **연결된 여러 자원과 시스템을 효율적으로 이용할 수 있는 소프트웨어와 시스템의 논리적인 설계가 반드시 필요**

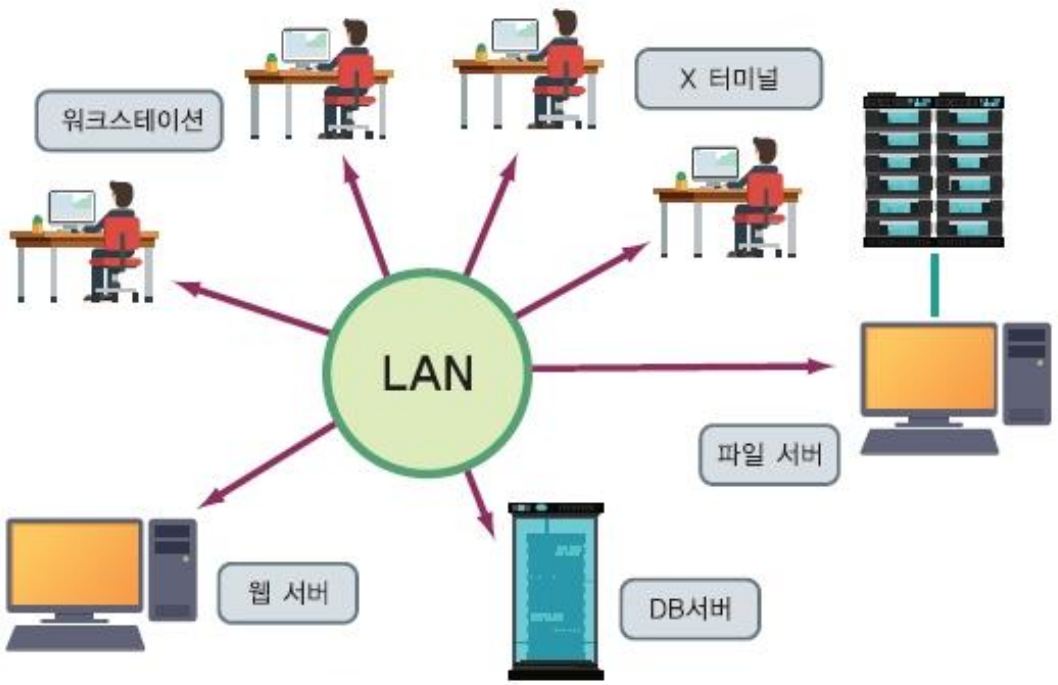


그림 5.17 ▸ 분산 처리 시스템의 개념

운영체제 운용 기법

구분	특징	예시	용도
배치 처리(Batch)	작업 모아서 순차 처리	IBM z/OS	대량 데이터 처리
다중 프로그래밍	CPU 자원 공유, 선점 없음	UNIX	서버·멀티태스킹
시분할(Time-Sharing)	짧은 시간 단위 선점 스케줄링	Linux	대화식 운영
실시간(Real-Time)	고정 응답 시간 보장	VxWorks	제어 시스템
분산(Distributed)	네트워크 통해 자원 통합	Windows Server	클러스터링
임베디드(Embedded)	특정 목적 전용 OS	RTOS, Android	IoT·가전제품

운영체제의 발전 과정

표 5.3 운영체제 발전 과정과 특징

세대	특징	의미
0세대 (1940년대)	• 운영체제가 없었음	• 기계어 사용
1세대 (1950년대)	• 일괄처리 시스템	• IBM 701용 OS • 단일 흐름 일괄 처리
2세대 (1960년대)	• 다중 프로그래밍 • 시분할 시스템 등장 • 실시간 처리 시스템	• 고급 언어로 운영체제 개발
3세대 (1960년대 중반~1970년 중반)	• IBM S/360, S/370 등장 • 다중모드 시스템 도입	• 일괄처리, 시분할 처리, 다중 처리를 하나의 시스템에서 제공(다중모드) • 유닉스 개발
4세대 (1970년 중반~1989)	• GUI 방식 발전 • 분산 처리 시스템 개념 • 개인용 컴퓨터 OS 등장	• 개인용 컴퓨터와 워크스테이션 등장 • TCP/IP의 등장
5세대 (1990년대~현재)	• 분산 처리 시스템 실현 • 지식 기반 시스템 등장 • 인공 지능 시스템 실현	• MS의 윈도우 강세 • 다양한 GUI 방식의 운영체제

MS-DOS의 출현

마이크로소프트의 MS-DOS는 Disk Operation System의 약자로 디스크 중심의 명령행 사용자 인터페이스(Command Line User Interface) 방식의 운영체제

- **1981년 8월 버전 1.0으로 시작**
  - 현재의 윈도우에서 명령어 쉘 기동
  - 명령 프롬프트를 실행
    - **DOS 기능을 수행하는 쉘(Shell)**
- **IBM에서 PC-DOS 개발**
  - 빌 게이츠(Bill Gates)와 폴 알렌(Paul Allen)을 고용
    - **도스 개발**
      - **디스크에 운영체제를 저장하여 이용**
  - IBM 호환 개인용 컴퓨터에 MS-DOS를 채용하게 되어 MS-DOS는 계속 발전
    - **마이크로소프트도 세계적인 소프트웨어 회사로 발전**

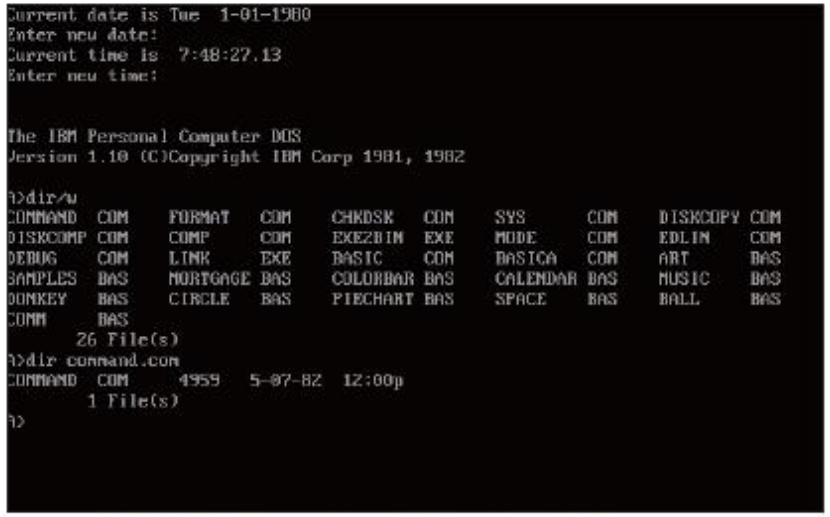


그림 5.18 ▸ 명령행 인터페이스 방식의 DOS 1.10

그림 5.19 ▸ DOS 1.0의 패키징

MS-DOS 발전과 경험

도스는 1981년 1.0에서 시작하여 1994년 MS-DOS 6.22까지 발전

- **명령어 중심의 운영체제**
- **쉘(shell)인 명령 프롬프트**
  - 명령어 cmd를 누르면 DOS 기능을 수행

표 5.4 MS-DOS 명령어

명령어	내용
cd	디렉토리 이동
dir	현재 디렉토리 정보 보기
copy	파일 복사
del	파일 삭제
edit	파일 편집
move	파일 이동
ren	파일 이름 수정
cls	도스 프롬프트에서 모든 내용 제거

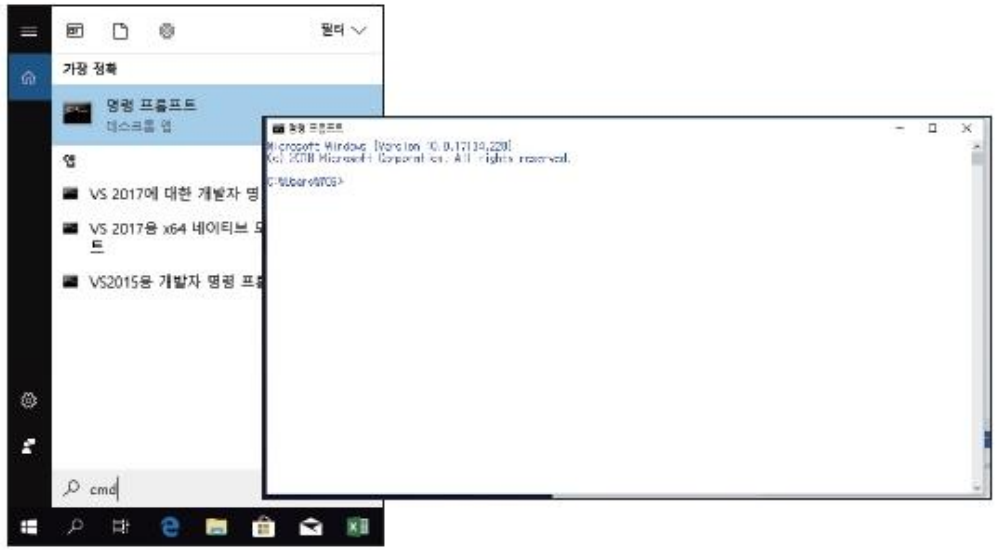


그림 5.20 ▸ 윈도우10에서 DOS 명령 쉘 실행

윈도우

도스 운영체제의 명령행 인터페이스 방식에서 발전하여 그래픽 사용자 인터페이스(GUI) 방식을 채택한 운영체제

- **GUI 방식**
  - 1983년 처음 출시
  - 1993년 3.1에서 널리 사용
    - **윈도우 3.1도 운영체제라기 보다는 운영환경에 불과한 것이었고, 도스가 실질적인 운영체제**
- **윈도우 95: 실질적인 GUI 운영체제**
    - **실질적인 그래픽 사용자 인터페이스를 채택한 하나의 독립된 운영체제**

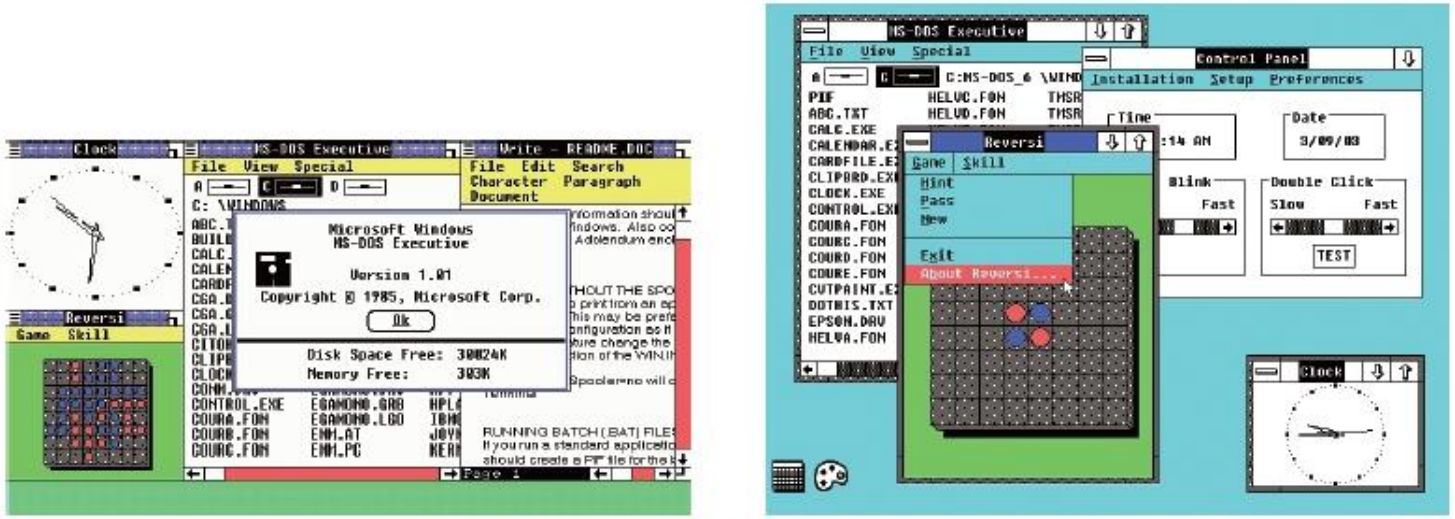


그림 5.22 ▸ MS의 윈도우 1.01(1985)과 윈도우 2.03(1987) 화면

MS 윈도우 발전

아직까지 개인용 컴퓨터의 대부분은 윈도우 사용

- **윈도우 10**
  - 2015년 윈도우 8의 차기 버전으로 발표
    - **코드명 '스레숄드'의 윈도우 10**
  - 가상 데스크톱 제공
    - **바탕 화면을 여러 개 생성 기능**
- **윈도우 11**
  - 2021년 10월에 출시
    - **CPU는 2개 이상 코어가 있는 1GHz 이상 65 비트 프로세서급**
    - **램(RAM)은 4GB, 저장공간은 64GB 이상인 시스템 사양**
  - 스마트폰의 안드로이드 앱을 내려 받고 실행
  - 협업도구 팀스(teams)가 기본 탑재
  - 화면 분할 기능 '스냅'을 제공
  - '다이렉트 스토리지'를 지원
    - **데이터 고속 처리 기능**

OS/2

IBM과 마이크로소프트 사가 협력하여 개발한 인텔의 80386/80486 계열 CPU에서 능력을 최대로 발휘하도록 설계한 운영 체제

- **Operating System/2**
  - IBM과 마이크로소프트 사가 협력하여 개발
    - **1987년 인텔의 80286 CPU에 적합하도록 16 비트 운영체제 OS/2를 개발**
    - **이후 인텔의 80386/80486 계열 CPU에서 능력을 최대로 발휘하도록 설계한 운영제제**
  - 1991년 IBM과 마이크로소프트는 결별하고 각각 독자적인 운영체제를 개발
    - **IBM은 독자적으로 OS/2를 발전시켜 인텔의 80386 CPU에 적합하도록 완전한 32비트 운영체제인 OS/2 2.0 발표**
      - **1992년에 발표**
    - **마이크로스프트가 OS/2를 발전시킨 것이 윈도우즈 NT**

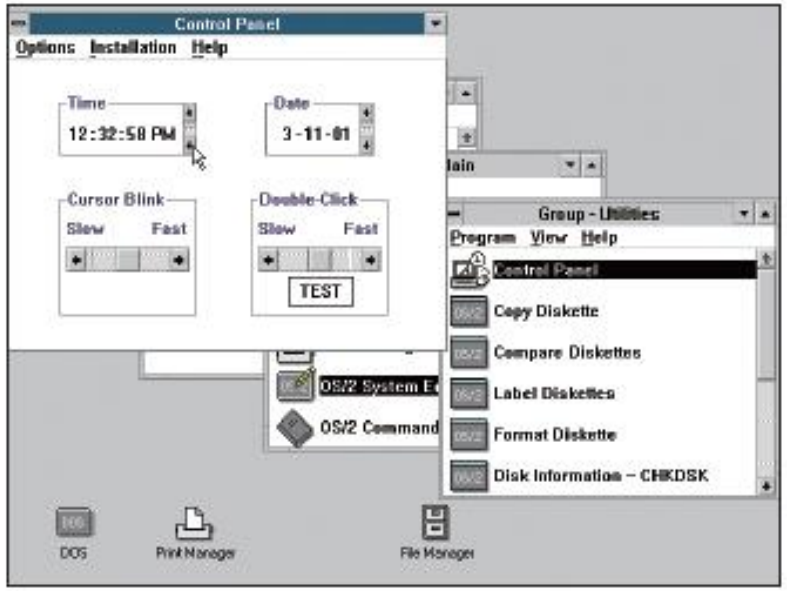
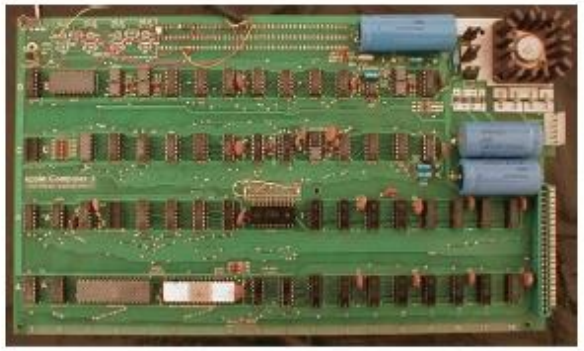


그림 5.32 ▸ OS/2 1.30(1990)의 화면

애플 컴퓨터

스티브 워즈니악(Steve Wozniak)과 스티브 잡스(Steve Jobs)는 1976년 애플(apple) 컴퓨터를 창설하고 애플 1을 발표

- **애플 I**
  - 개인용 컴퓨터로 처음 발표된 컴퓨터
  - 1977년에는 이를 발전시킨 애플 II를 발표



기판

잡스와 워즈니악

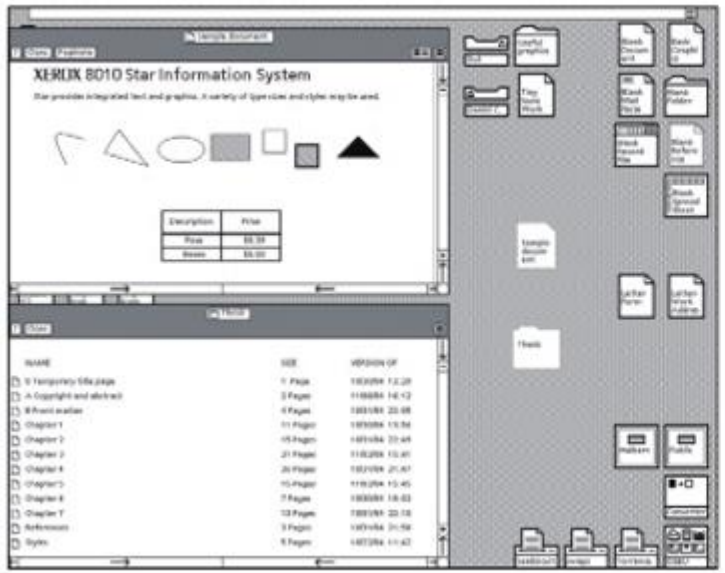
컴퓨터

그림 5.34 ▸ 애플 컴퓨터의 애플 I의 기판과 컴퓨터

제록스 스타 워크스테이션과 애플의 리사

최초의 GUI 방식 운영체제 탑재

- **제록스 스타 워크스테이션(Star Workstation)**
  - 1981년 제록스(Xerox) 사는 그래픽 운영체제의 모체가 된 그래픽 운영환경을 채택해 발표
- **애플 컴퓨터의 리사**
  - 1983년에 처음으로 그래픽 사용자 인터페이스 방식의 운영체제를 장착한 리사(Lisa)를 발표
  - 일반 비즈니스 시장을 목표로 한 최초의 개인용 컴퓨터
    - **최초로 그래픽 사용자 인터페이스의 운영체제를 탑재한 개인용 컴퓨터**



애플의 운영체제 시스템

애플은 1984년 상업용 개인 컴퓨터인 매킨토시(Macintosh)를 발표

- **매킨토시**
  - 그래픽 사용자 인터페이스 방식의 운영체제 시스템 1(System 1)을 사용
  - 시스템 1 운영체제
    - **당시 명령행 방식의 MS-DOS보다 매우 앞선 기술이었으며 운영체제를 한 단계 향상시키는 계기**
    - **애플의 운영체제는 시스템 1에서 시스템 7까지 버전 향상이 이루어짐**
  - 1997년에는 이름을 바꾸어 맥 OS 8(Mac OS 8)로 발전

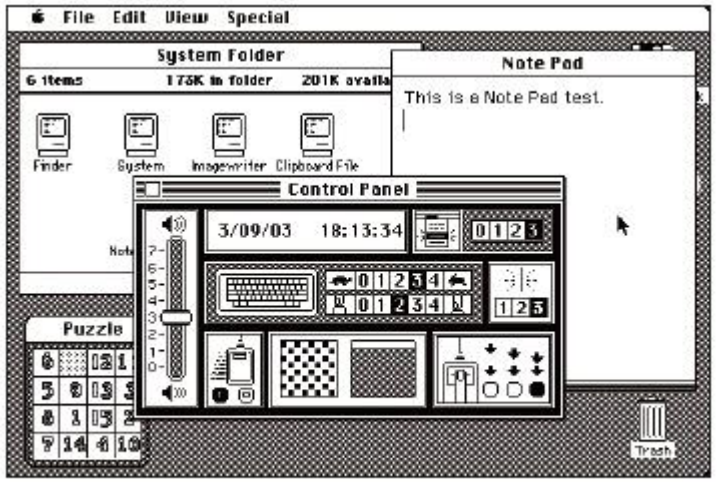


그림 5.37 ▸ 애플 매킨토시 I의 운영체제 시스템 1(1984)

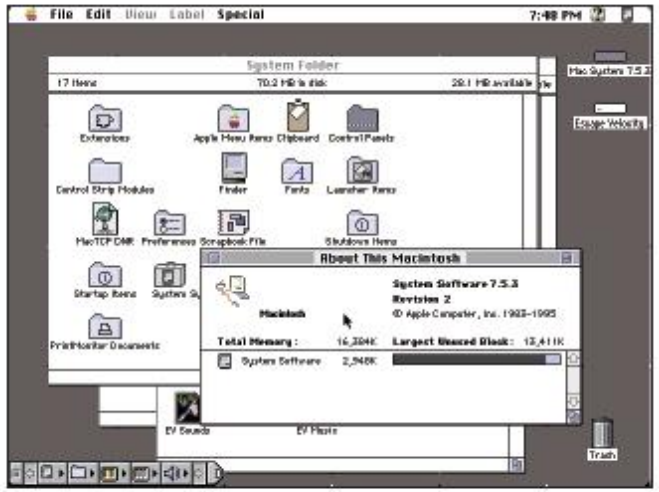


그림 5.38 ▸ 애플 매킨토시의 운영체제 시스템 7(1991)과 맥 OS 8 (1997)

애플의 운영체제 OS X

2001년 애플의 운영체제는 OS 9을 클래식환경이라는 이름으로 명명하여 OS X로 대변신을 시도

- **맥 OS X 10**
  - 유닉스(Unix)에 기반한 운영체제
  - 유닉스 환경을 배경으로 하는 운영체제인 넥스트(NeXT)의 운영체제를 기본
- **맥 OS X 11**
  - OS X 요세미티(Yosemite)는 애플의 11번째 OS X 운영체제
  - 2014년 가을에 정식 출시
    - **애플의 필수적인 요소는 유지하면서 강력하고 사용하기 쉬운 인터페이스를 추구**
  - 모바일 운영체제인 iOS와의 쉽게 연동
    - **데스크톱인 맥(Mac)과 아이패드, 아이폰 사이에서 여러 작업이 가능**

그림 5.42 ▸ macOS 모하비 화면

리눅스

핀란드의 리누스 토발즈(Linus Torvalds)에 의하여 개발된 유닉스(Unix) 기반의 운영체제

- **1991년 11월 버전**
  - 개인용 컴퓨터에 적합한 공개용 운영체제
  - 리눅스의 소스 코드를 일반공중허가(GPL: General Public License)
    - **인터넷에 공개해 모든 사람이 자유롭게 사용**
  - 일반인들이 직접 리눅스의 개선에도 참여
- **특징**
  - 인터넷 프로토콜인 TCP/IP를 강력하게 지원하는 등 네트워크 기능이 강화
    - **다중 사용자(Multi User)**
    - **다중 프로그래밍(Multi Programming) 방식을 지원**
    - **안정성 있는 운영체제**
- **다양한 버전의 리눅스**

유닉스

유닉스(Unix)는 벨 연구소의 데니스 리치(Dennis M. Ritchie)와
켄 톰슨(Ken Thompson)이 참여한 멀틱스(MULTICS) 운영체제 팀에서 개발한 운영체제

- **UNICS**
  - UNiplexed Information and Computing Service
  - PDP-7 새로운 시스템의 운영체제로 개발
- **UNICS가 발전되어 UNIX로**
  - 1973년 새로 개발한 언어 C를 이용
    - **시스템 PDP-11을 위해 만든 운영체제가 유닉스(Unix)**
  - 최초 C 언어로 작성된 소스 코드는 약 11000 줄
    - **95%인 10000 줄은 C 언어로 작성**
      - **나머지 1000 줄은 어셈블리 언어로 작성**
    - **이식성(Portability)이 좋은 운영체제**

그림 5.48 ▸ PDP-11 앞에서 유닉스 개발자 켄 톰슨과 데니스 리치

유닉스 운영체제의 발전과 구성 요소

유닉스 운영체제는 대학과 연구소를 중심으로 교육용과 연구용으로 널리 사용

- **벨 연구소는 유닉스의 소스 코드를 공개**
  - 시스템 V(System V)
    - **벨 연구소에서 개발한 유닉스는 계속 발전**
  - BSD(Berkeley Software Distribution) 1977년
    - **버클리 대학(the University of California at Berkeley)**
      - **컴퓨터 연구소 그룹(CSRG: Computer System Research Group)에서 개발한 유닉스 계열의 운영체제**
  - 중대형 컴퓨터 회사에서 다양하게 발전
- **커널(Kernel)**
  - 운영체제의 가장 핵심이 되는 모듈
    - **CPU와 주기억장치를 관리하고 시스템 호출을 처리하며 주변장치를 관리하는 기능**
- **쉘(Shell)**
  - 도스와 같은 명령행 인터페이스 방식
    - **순수 유닉스 운영체제와 사용자 사이에서 중간적인 매개 역할을 하는 프로그램**
    - **표준 UNIX 명령 인터프리터**
    - **사용자가 입력한 명령을 해석하여 그에 맞는 다른 프로그램을 실행시키는 유틸리티 프로그램**
- **종류**
  - 스티븐(Steven Bourne)의 본쉘 (Bourne Shell, sh),
  - 빌 조이(Bill Joy)의 C 쉘(csh),
  - 데이브 콘(Dave korn)의 콘쉘(Korn Shell, ksh)